GUIDE 33 · OPERATE

Back up and restore the store

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=33-back-up-and-restore. This copy was made on 4 November 2026.

Time: 15 minutes, including a practice restore. **You'll end up with:** a consistent backup taken while the server runs, a restore you've actually tried, and a list of what the backup doesn't hold.

Everything QueryAPIGate owns is in its **store**: connections, saved queries and their versions, run history, API keys (as hashes), roles and the audit log. By default that's one SQLite file, `queryapigate.db`, in `QUERYAPIGATE_HOME`; with `QUERYAPIGATE_DATABASE_URL` it's a schema in PostgreSQL.

The SQLite store

Back up - with `queryapigate backup`, not `cp`

```shell
QUERYAPIGATE_HOME=/srv/queryapigate queryapigate backup /backups/queryapigate-2026-10-04.db
# Backed up /srv/queryapigate/queryapigate.db to /backups/queryapigate-2026-10-04.db (204800 bytes)
```

It's safe while the server is serving: SQLite's own backup API copies one consistent moment, **including recent changes still in the write-ahead log** (`queryapigate.db-wal`). A plain `cp queryapigate.db` misses those. Verified on a running server - the two copies, taken moments apart:

	AUDIT ENTRIES	API KEYS	RUNS IN HISTORY
`cp queryapigate.db`	49	30	220
`queryapigate backup`	51	32	243

The `cp` copy was missing the last two keys created. An existing target file is refused unless you pass `--force` (exit status 2), so a scheduled job with a dated name never overwrites yesterday's backup by accident.

With Docker:

```shell
docker compose exec queryapigate queryapigate backup /data/backup.db --force
docker compose cp queryapigate:/data/backup.db "./backups/queryapigate-$(date +%F).db"
docker compose exec queryapigate rm /data/backup.db
```

Schedule it like any job - see Run a scheduled export for cron and systemd - and copy the result off the machine.

Restore

Stop the server, put the backup in place as `queryapigate.db`, **delete the old `-wal` and `-shm` files** (they belong to the store you're replacing, and SQLite would try to apply them), and start it:

```shell
systemctl stop queryapigate          # or: docker compose stop queryapigate
cp /backups/queryapigate-2026-10-04.db /srv/queryapigate/queryapigate.db
rm -f /srv/queryapigate/queryapigate.db-wal /srv/queryapigate/queryapigate.db-shm
systemctl start queryapigate
```

Verified into a fresh `QUERYAPIGATE_HOME`: the restored server listed exactly the same API keys as the original, and existing scoped keys authenticated with their old secrets.

The PostgreSQL store

Use `pg_dump`, as for any PostgreSQL data. `queryapigate backup` tells you the exact command, with the store's schema filled in, rather than copying anything itself:

```shell
queryapigate backup /backups/queryapigate.dump
# queryapigate: the store is PostgreSQL - back it up with pg_dump, as the rest of that database (see
DEPLOYMENT.md, Backups and restores):
#   pg_dump --format=custom --schema=qag --no-owner --no-privileges --file=/backups/queryapigate.dump
"$QUERYAPIGATE_DATABASE_URL"
```

`pg_dump` reads one snapshot, so it's safe while QueryAPIGate runs. To restore: stop every QueryAPIGate process using the store, `pg_restore --no-owner --no-privileges --dbname="$QUERYAPIGATE_DATABASE_URL"` `queryapigate.dump` into an empty database (or after dropping that schema), and start them again. If your managed database's snapshots already cover the whole database, they cover the store too. Details: DEPLOYMENT.md.

What a backup doesn't contain

The practice restore above found two of these the hard way - a query on a SQLite connection failed with *SQLite database file not found*, and one on a PostgreSQL connection with *Environment variable 'WAREHOUSE_PASSWORD' referenced by the connection is not set*:

NOT IN THE BACKUP	WHAT TO DO
The databases QueryAPIGate connects to - including SQLite and DuckDB files in `QUERYAPIGATE_HOME`	back them up in their own right
Environment variables connections refer to (`${WAREHOUSE_PASSWORD}`)	keep them with your other secrets
`QUERYAPIGATE_SECRET_KEY`	without it, encrypted passwords can't be read - keep it in your secret manager, **not** next to the backup
`QUERYAPIGATE_API_KEY` (the admin key)	it's configuration, not data
Scoped API key secrets	only hashes are stored: keys keep working after a restore, but a lost secret can't be recovered from a backup
The Redis response cache	nothing to do - it refills

Restoring into a different version

A backup from an **older** release is upgraded automatically on first start, like any upgrade. One from a **newer** release is refused with a message, rather than run by code that doesn't understand it. So restoring yesterday's backup after an upgrade is fine; going back to an older release means restoring a backup taken before the upgrade. See Upgrade to a new version safely.

Practise it

A backup nobody has restored is a hope, not a backup. Once, and after any big change: restore the latest backup into a scratch `QUERYAPIGATE_HOME` on another port, and check that the connections, keys and saved queries you expect are there:

```shell
mkdir /tmp/restore-test && cp /backups/queryapigate-2026-10-04.db /tmp/restore-test/queryapigate.db
QUERYAPIGATE_HOME=/tmp/restore-test QUERYAPIGATE_API_KEY=... queryapigate serve --port 5099
curl http://127.0.0.1:5099/api/v1/api-keys -H 'X-API-Key: ...'
```

Next steps

- Upgrade to a new version safely - always after a backup.
- Keep connection passwords out of the store.
- Deploy with Docker for real.